THE UNIVERSITY OF AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

MODELLING USING UML - I

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

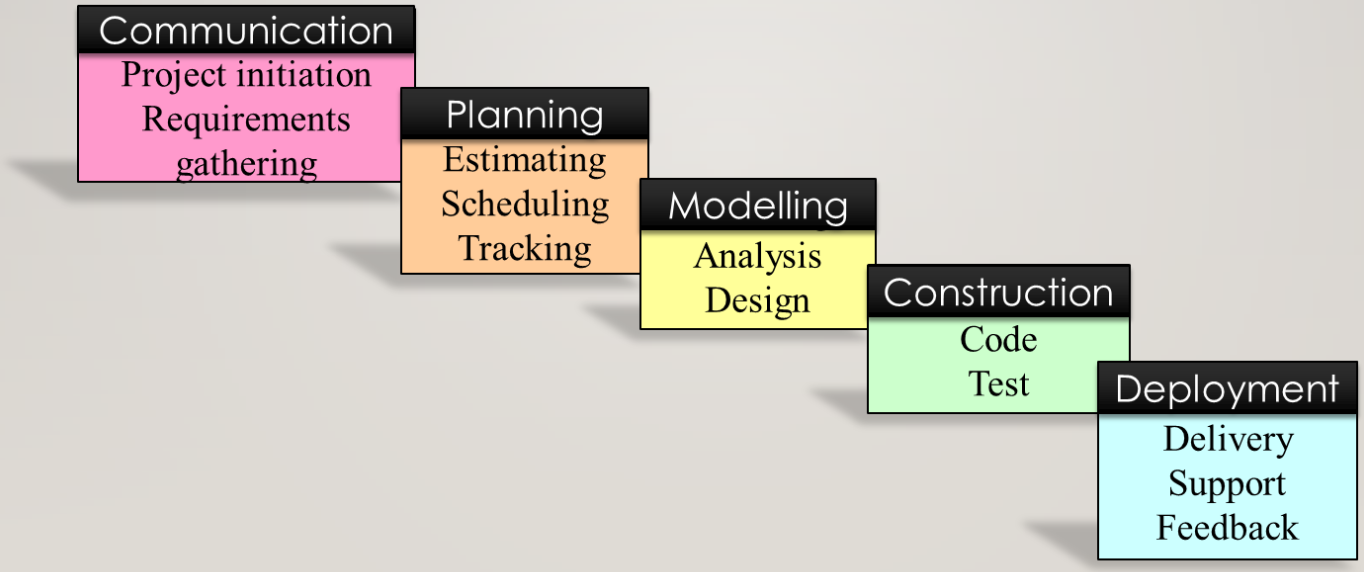
PARAMVIR SINGH

AGENDA

- The Larger Picture
- A Real World Analogy
- Modelling in Engineering
- The Unified Modelling Language (UML)
  - Multiple views
  - Class diagrams
- *Exercise*
  - Modelling Shapes application

THE LARGER PICTURE

SOFTWARE ENGINEERING PROCESS



Software Development Life Cycle Phases

FROM THE REAL WORLD – MODELLING A HOUSE

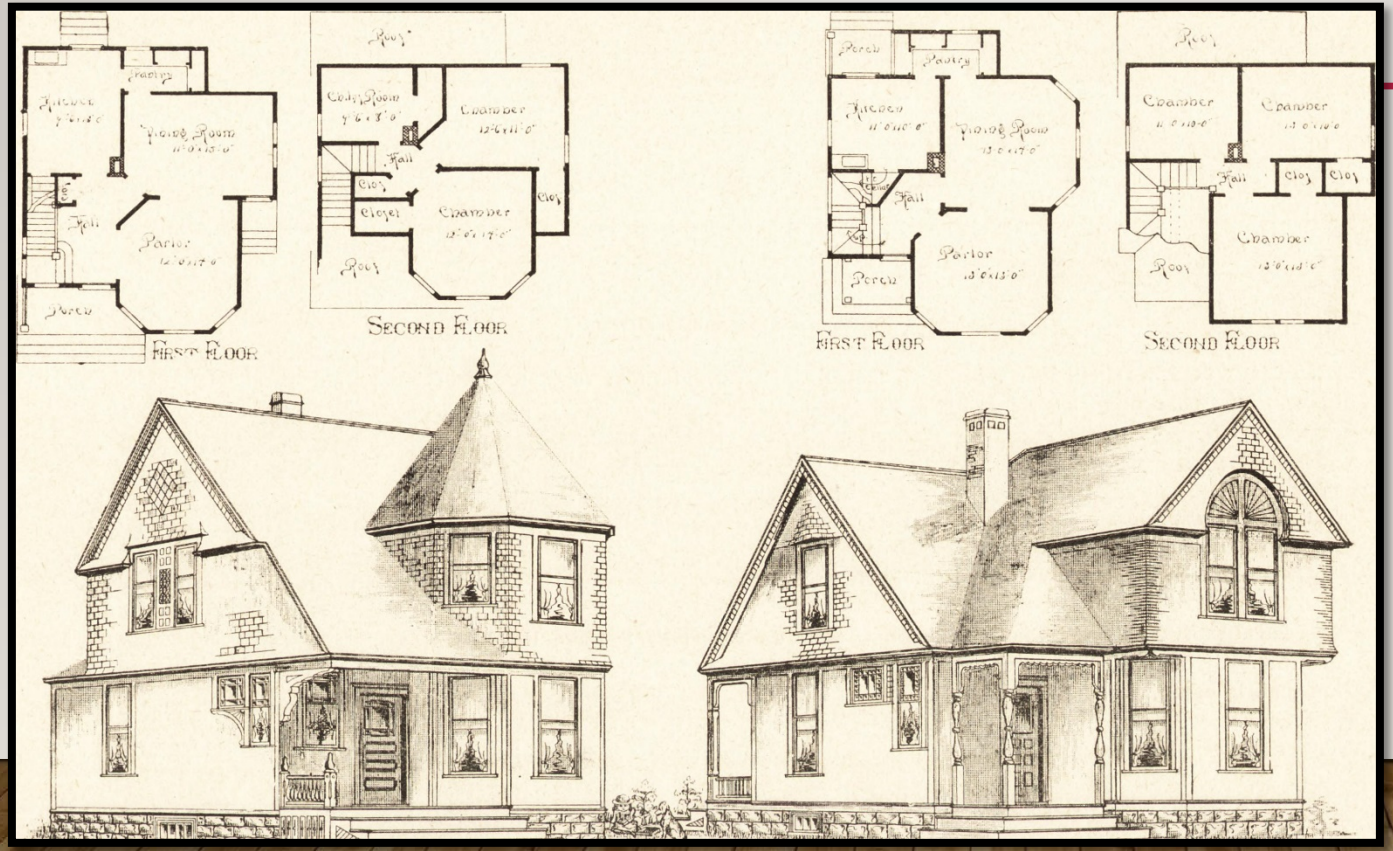
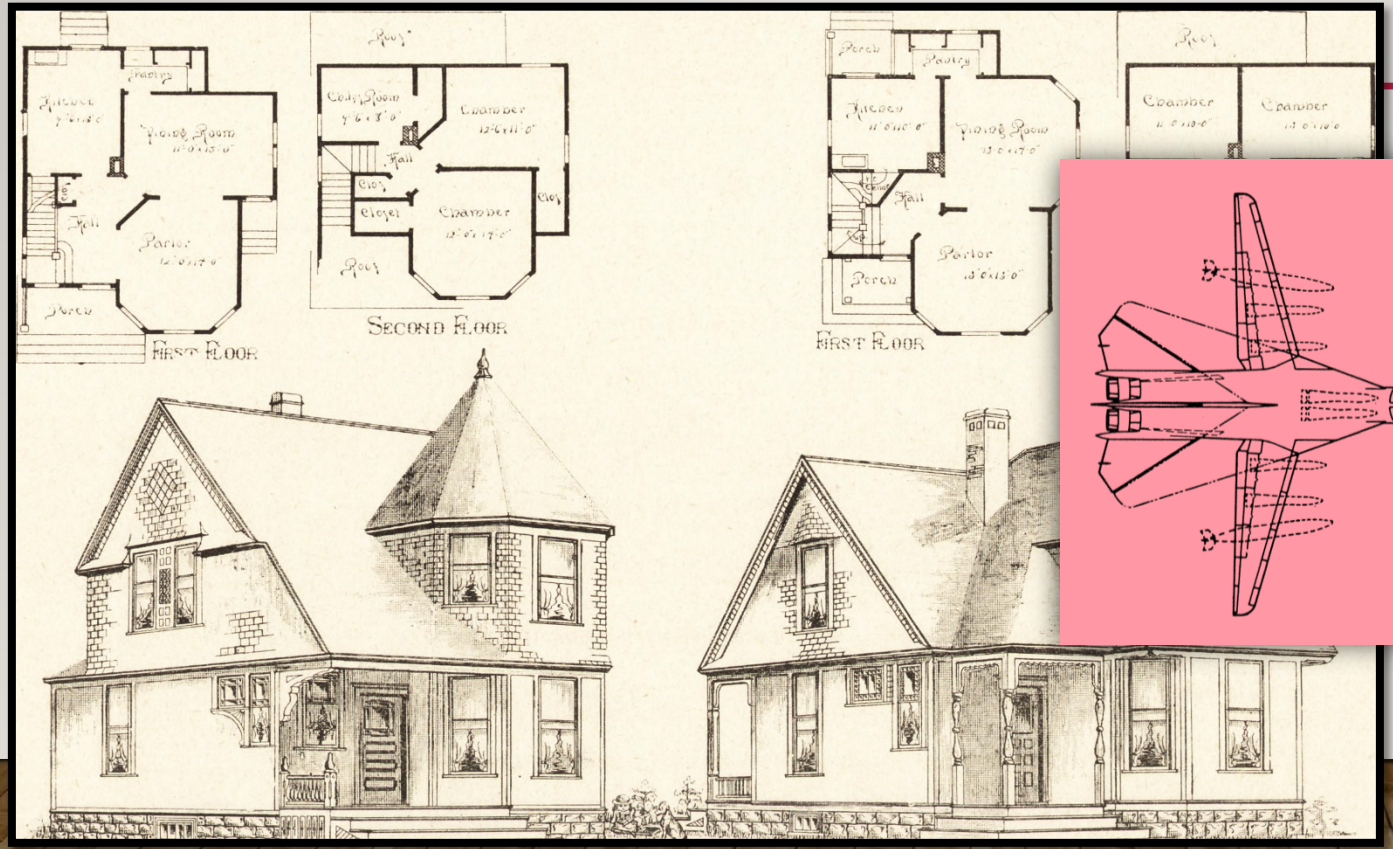


Image Source - https://commons.wikimedia.org/wiki/

FROM THE REAL WORLD – MODELLING A HOUSE/AEROPLANE



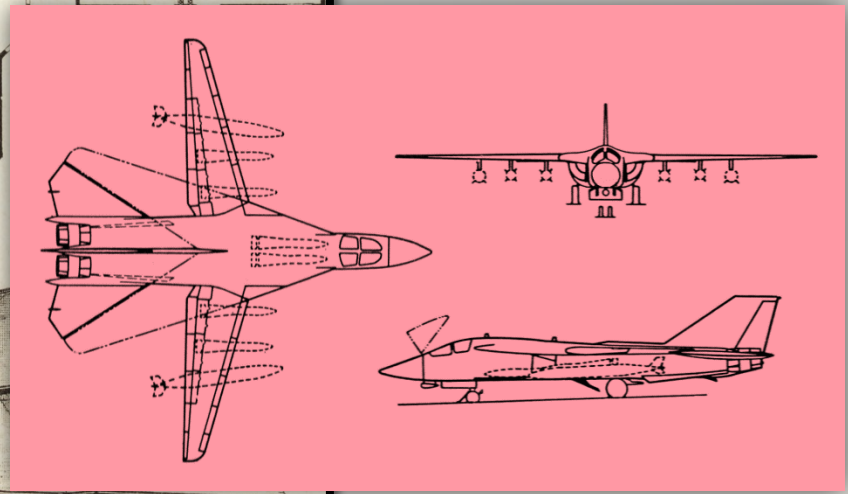
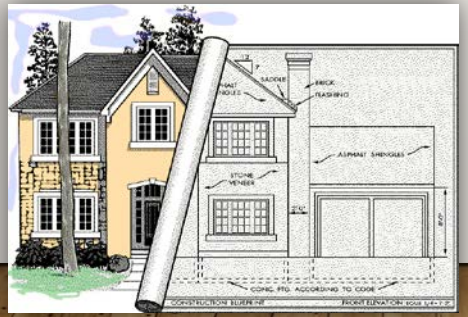


Image Source - https://commons.wikimedia.org/wiki/

Image Source:https://commons.wikimedia.org/wiki/File:General_Dynamics_F-111B_3-view_drawing.png

MODELLING IN ENGINEERING

- **Key modelling questions**
  - What is a model?
  - Why do we model?

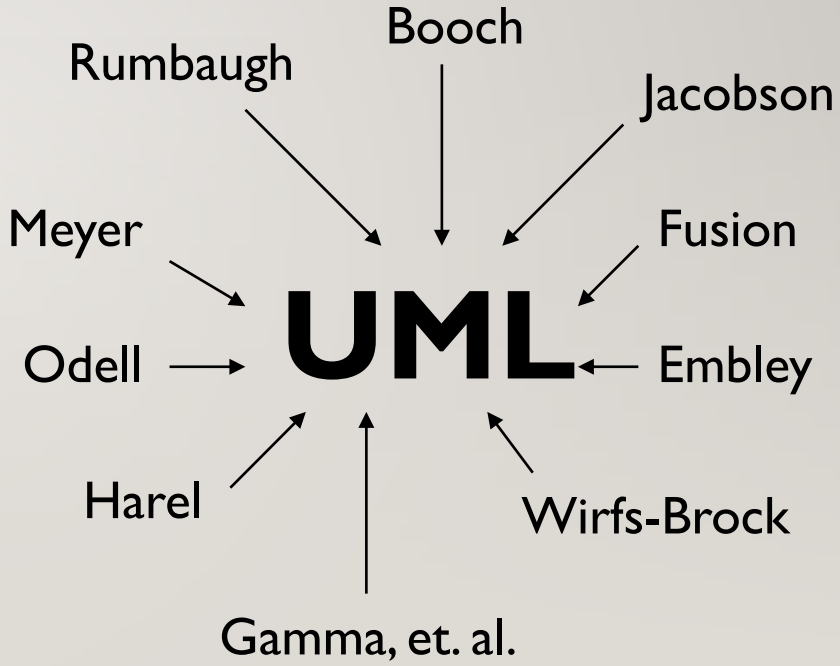


- **Four principles of modelling**
  (Booch 1999)

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is changed
2. Every model may be expressed at different levels of precision
3. The best models are connected to reality
4. No single model is sufficient. Every non-trivial system is best approached through a small set of nearly independent models

THE UNIFIED MODELLING LANGUAGE (UML)

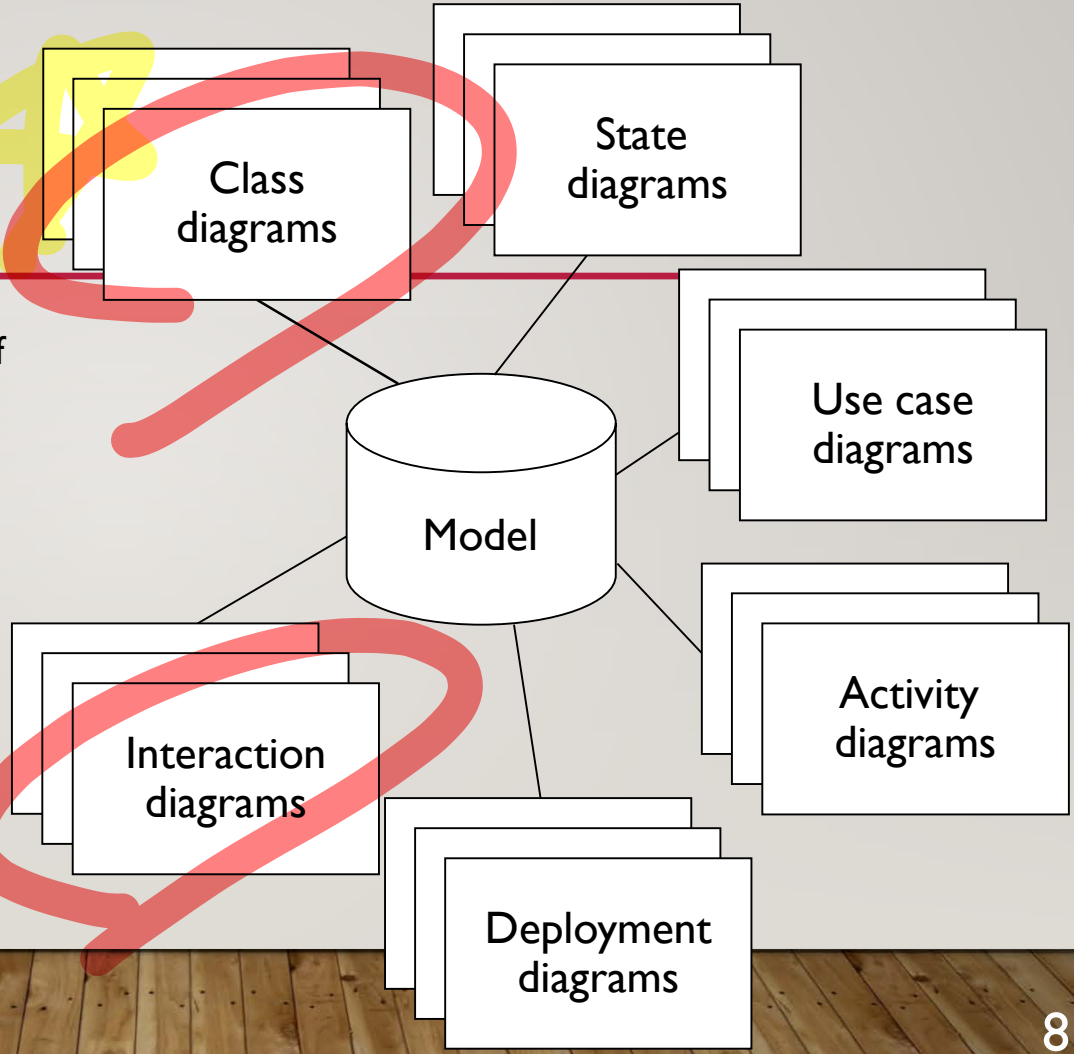
- The UML is a **standard** notation for writing software blueprints
- The UML can be used to model software-intensive systems
- The UML is a fusion of concepts and notations from other methodologists



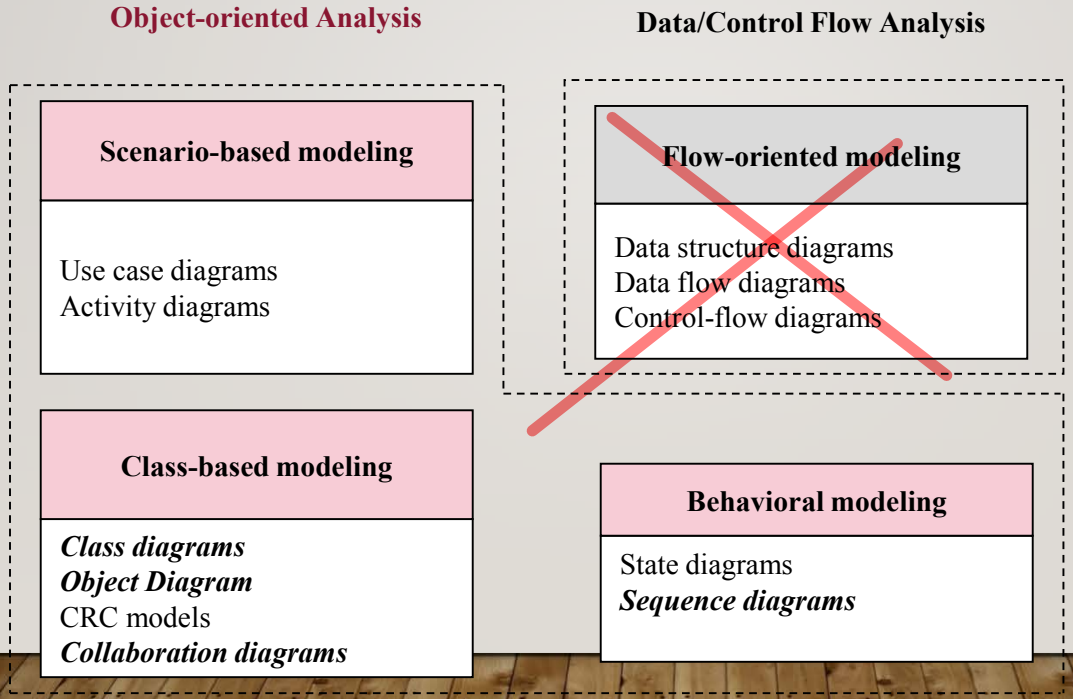
https://en.wikipedia.org/wiki/Unified_Modeling_Language

UML DIAGRAMS

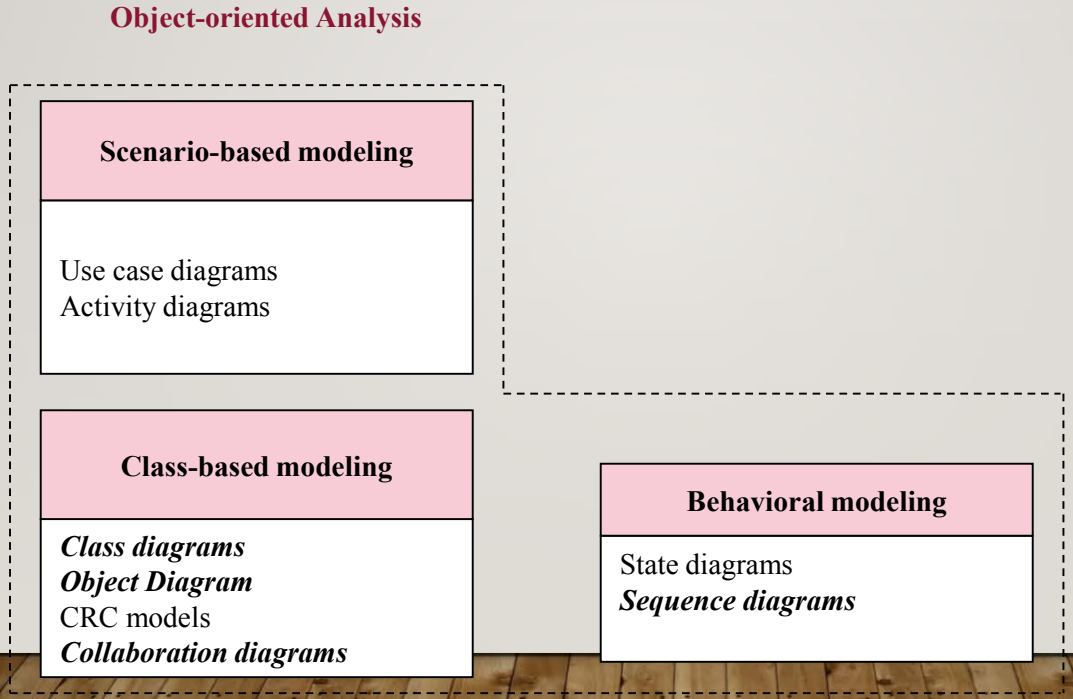
- The UML is used to construct a model of a software system
- A model is a simplification of reality
- A model is thus an **abstraction** that exposes interesting properties while suppressing unnecessary detail
- A UML model comprises different diagrams, each focusing on a particular **view** (aspect) of a system

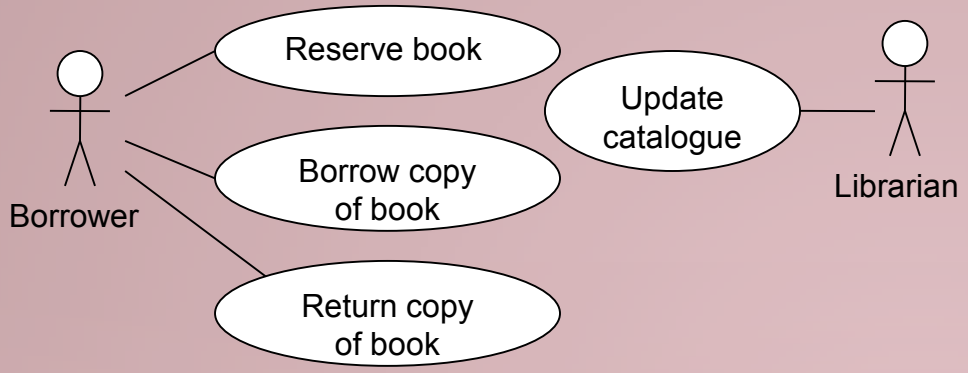


UML DIAGRAMS

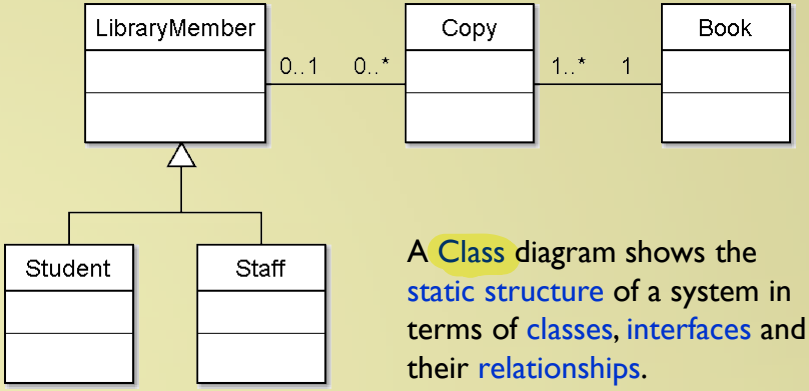


UML DIAGRAMS

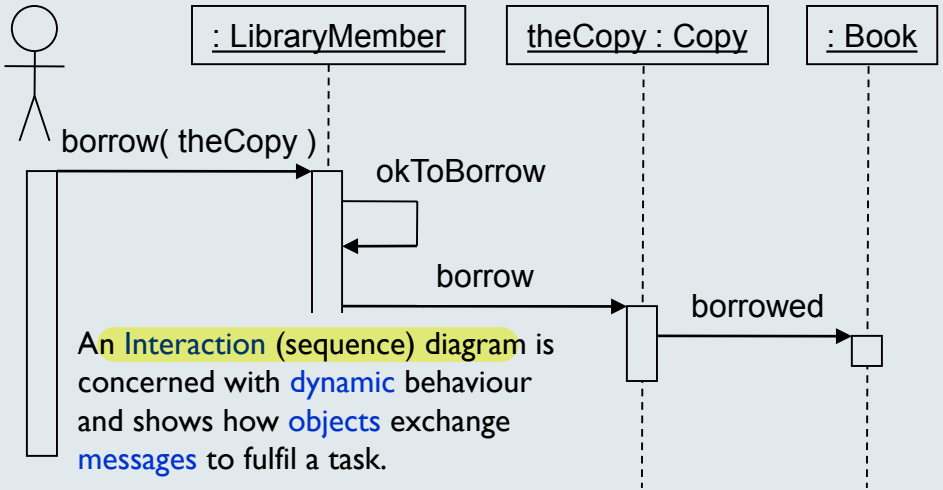




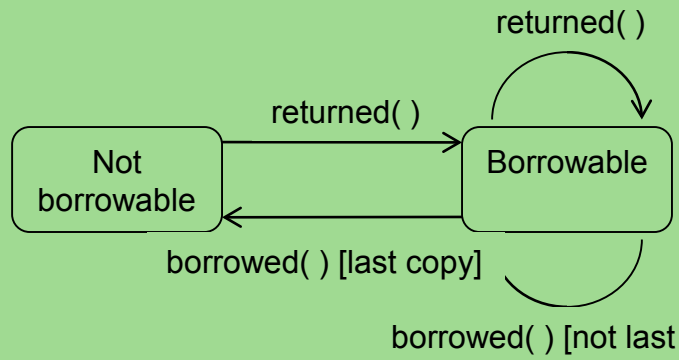
A Use Case diagram is user-centered and identifies system users (actors) and tasks.



A Class diagram shows the static structure of a system in terms of classes, interfaces and their relationships.



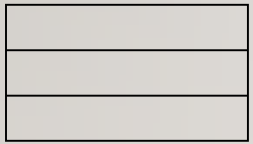
An Interaction (sequence) diagram is concerned with dynamic behaviour and shows how objects exchange messages to fulfil a task.



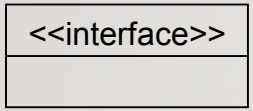
A State diagram shows the states and transitions for an instance of particular class (Book in this case).

CLASS DIAGRAM NOTATION

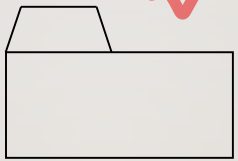
- Structural elements



Class

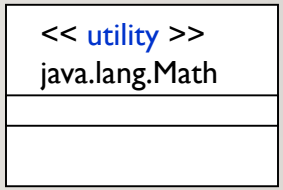


Interface

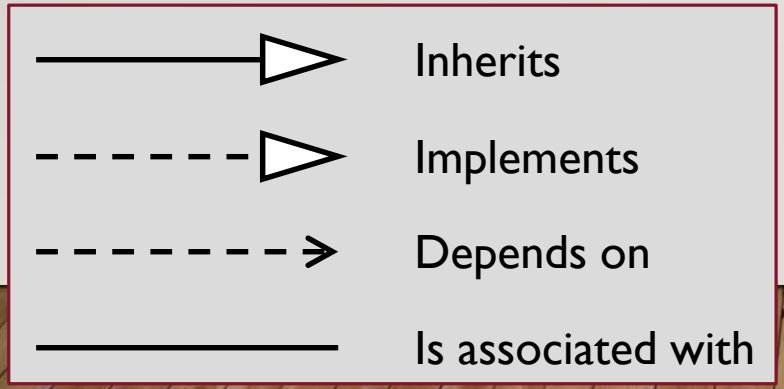


Package

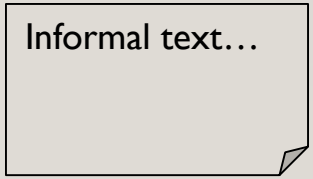
- Stereotyped classes



- Relationships

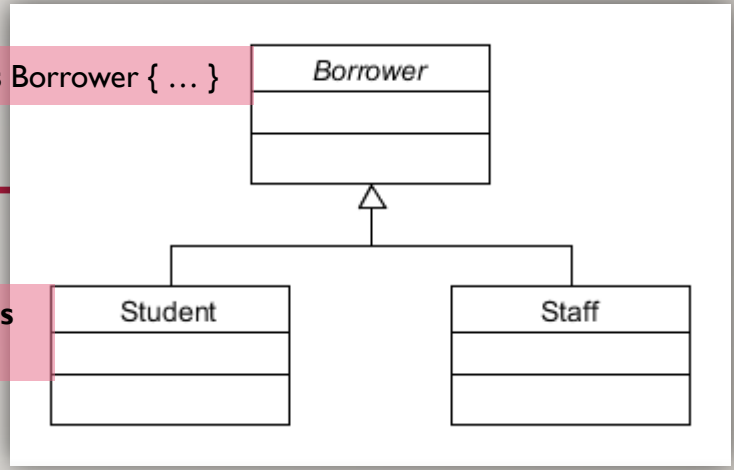


- Annotations



CLASS DIAGRAMS

abstract class Borrower { … }



class Student **extends** Borrower { … }

Book
- title : String
+ copiesOnshelf() : int + borrow(c : copy) : void
Responsibilities -- Represents a book that can be borrowed.

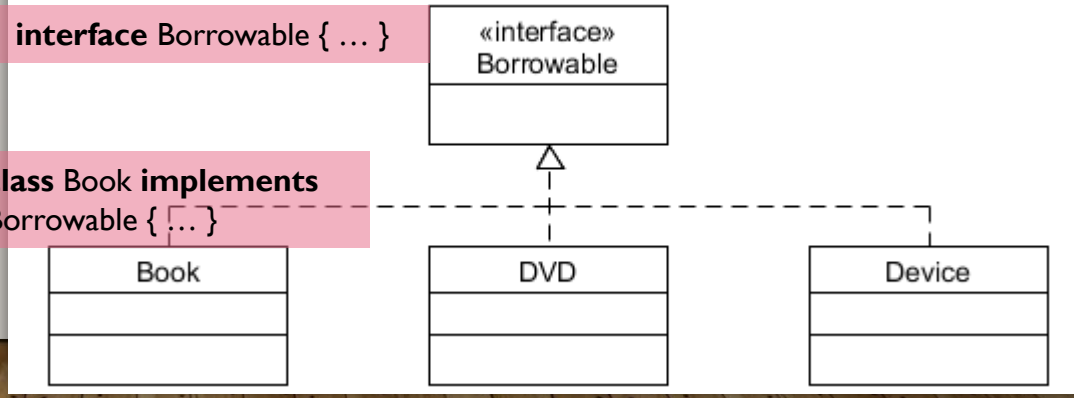
```
public class Book {
   private String title;

   public int copiesOnshelf( ) {
      …
   }

   public void borrow(Copy c) {
      …
   }
}
```

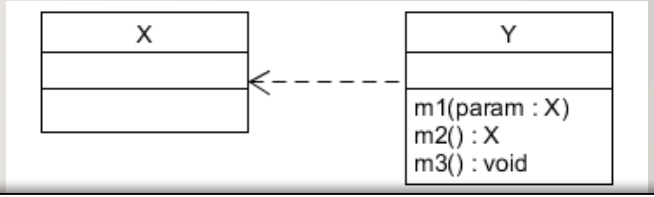
interface Borrowable { … }

class Book **implements** Borrowable { … }



DEPENDENCIES

- A **dependency** is used to show that one class uses another
- The link is directed towards the used class
- A dependency means that a change to the used class (X) might necessitate a change to the other class (Y)




```
public class Y {
   …
   public void m1( X param ) {
      …
   }

   public X myMethod2( ) {
      …
   }

   public void myMethod3( ) {
      X localVar …
   }
}
```

Method argument

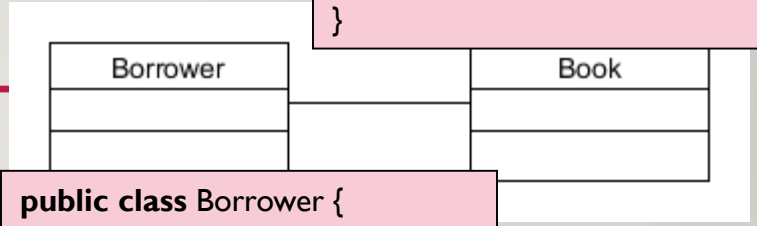
Method return type

Local variable

ASSOCIATIONS

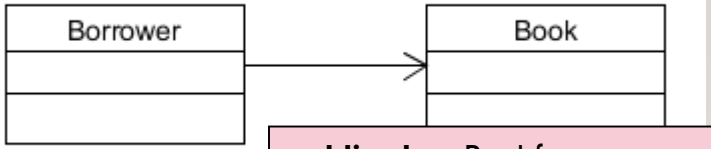
- An association is a **key dynamic relationship** between objects
- When two classes are associated, a pair of instances store references to each other (**bi-directional association**)
- Associations are by default bidirectional, but **can be unidirectional**

```
public class Book {
   private Borrower _onLoanTo;
   …
}
```




```
public class Borrower {
   private Book _bookOnLoan;
   …
}
```

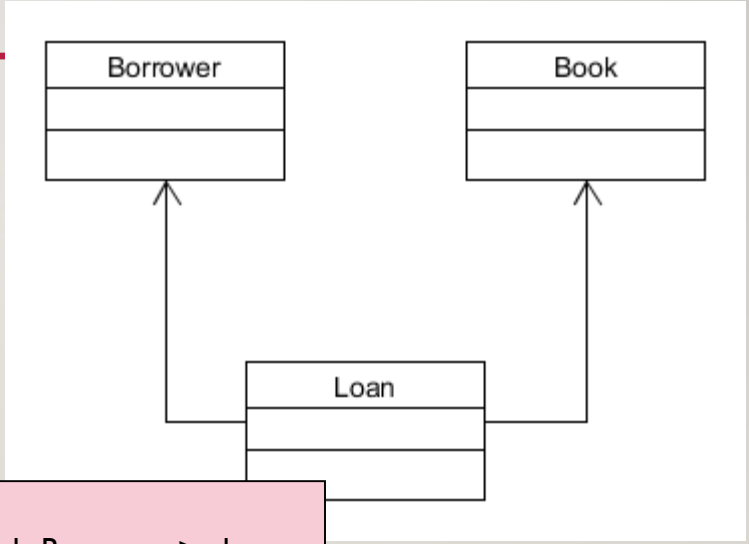
```
public class Borrower {
   private Book[ ] _booksOnLoan;
   …
}
```




```
public class Book{
   // No reference to Borrower
   …
}
```

ASSOCIATION CLASS

- Where desired, a bidirectional association can be broken using a **link or association class**
- The reduced coupling of this design promotes reuse




```
public class Loan {
   private Map< Book, Borrower > _loans;
   …
}
```

IN-CLASS EXERCISE

DRAW A CLASS DIAGRAM FOR SHAPES APP CODE

```
public abstract class Shape {
   protected int _x;
   protected int _y;
   protected int _deltaX;
   protected int _deltaY;
   protected int _width;
   protected int _height;
   …

   public void move( int width, int height ) {
      …
   }

   public abstract void paint( Painter p );
}
```

```
public class RectangleShape extends Shape {
   …
   public void paint( Painter p ) {
      …
   }
}
```

```
public class OvalShape extends Shape {
   …
   public void paint( Painter p ) {
      …
   }
}
```

```
public interface Painter {
   …
   void drawRect( … );
   void drawLine( … );
   void drawOval( … );
}
```

```
public class GraphicsPainter implements Painter {
   private Graphics _g;
   …
}
```

```
public class MockPainter implements Painter {
   …
}
```

```
public class AnimationViewer extends JPanel
      implements ActionListener {
   private List<Shape> _shapes;
   private Timer _timer = …
   …

   public void paintComponent( Graphics g ) {
      …
   }

   public void actionPerformed( ActionEvent e ) {
      …
   }

   public static void main( String[ ] args ) {
      JFrame frame = …
      …
   }
}
```

SUGGESTED TASKS

- Develop two versions of basic UML class model for the **A2** application
  - One for the **A2** skeleton/given code
  - Another one for your **A2** solution (i.e. from the code resulting after you have finished A2 assignment)
- Discuss the changes you observed in the two class diagrams.

REVIEW

- **Learnt something about:**
  - The what and why of modelling
    - Models are abstractions of complex systems; they expose significant information and supress irrelevant detail
    - Models help us to capture design decisions, to understand existing systems and to communicate design knowledge to others
  - The UML – the industry standard notation for modelling software-intensive systems
    - The UML includes diagram notations for modelling different aspects of software systems including static and dynamic structures (using class and object interaction diagrams respectively)
  - Models should be well-formed – individual views should be consistent with one another, resulting in a coherent model

REFERENCES

- **[Books]**
  - Chapter-5 Modelling with Classes, Timothy Lethbridge and Robert Leganaire, Object-Oriented Software Engineering: Practical Software Development using UML and Java, McGraw-Hill publications
  - *Java How to Program*, Dietel and Dietel Publications
- **[Online]**
  - Canvas
    - A2 code
  - Unified Modelling Language
    - https://en.wikipedia.org/wiki/Unified_Modeling_Language

UP NEXT…

- **Object-oriented software modelling using UML-2**